

Security and Compliance for LLM Systems

Prompt Injection Defense · PII Handling · Tenant Isolation · Data Residency · Audit Trails ·
Model Supply Chain

WEEK 4 · DAY 25

Thursday, July 30 2026

Day 25 of 84

The Threat Model Is Genuinely New

THREE STRUCTURAL PROPERTIES THAT MAKE LLM SYSTEMS DIFFERENT

- 1. THE INSTRUCTION/DATA BOUNDARY DOES NOT EXIST.
In SQL you have parameterized queries: the query is code, the parameters are data, and the DATABASE enforces the line. An LLM has no equivalent. System prompt, retrieved schema, and user question are the same kind of thing to the model.
-> There is no "prepared statement" for prompts. This is why prompt injection is not solvable the way SQL injection was solvable.
- 2. THE SYSTEM WRITES AND EXECUTES CODE.
Day 21's capstone generates SQL and runs it against production. The attack surface is not "what can the user type" but "what can the user cause the model to WRITE."
- 3. THE OUTPUT IS NON-DETERMINISTIC.
A control you tested 100 times can fail on attempt 101 with the same input. Controls that depend on the model behaving correctly are not controls. They are hopes.

THE ARCHITECTURAL CONSEQUENCE OF ALL THREE:
Every real security control must sit OUTSIDE the model.
Anything implemented by asking the model nicely is a mitigation at best, and usually a placebo.

Sec	Topic	Coverage
Sec 1	Prompt Injection	Direct vs indirect; the capstone's concrete exposure
Sec 2	Defense in Depth	Layer 0 DB privilege → Layer 3 blast radius
Sec 3	PII Handling	Six accumulation paths, tokenization, log + cache hygiene
Sec 4	Tenant Isolation	IsolationTestSuite, five leak points, residency
Sec 5	Audit Trails	AuditRecord, hash chain, erasure vs immutability

Sec 6	Supply Chain	Model provenance, serialization risk, third-party prompts
Sec 7	FAANG Practice	Google SAIF / Microsoft / Amazon / Meta / Netflix
Sec 8	Architect's Lens	Control hierarchy Tier 1-4; assume injection succeeds
Sec 9	Exercises + Summary	Injection test, PII audit, isolation proof, worst case

Section 1 — Prompt Injection: Direct and Indirect

Prompt injection is the injection of instructions through a channel the system treats as data. Two variants, with very different risk profiles.

[illegible]

```
        r"(admin|developer|debug|god)\s+mode",
        r"repeat\s+(your|the)\s+(system\s+)?(prompt|instructions)",
        r"</?(system|instruction|prompt)>",          # tag-injection attempts
    ]
    EXFIL_PATTERNS = [
        r"select\s+.*\bfrom\s+(auth|credential|token|secret|password)",
        r"\bunion\s+(all\s+)?select\b",
        r"information_schema",
        r"pg_(catalog|shadow|authid|user)",
    ]

    def scan(self, text: str, source: str) -> InjectionSignal:
        hits = [p for p in self.INSTRUCTION_PATTERNS if re.search(p, text.lower())]
        hits += [p for p in self.EXFIL_PATTERNS if re.search(p, text.lower())]
        # RETRIEVED CONTEXT should NEVER contain imperative instructions.
        # A hit here is far more suspicious than a hit in a user question,
        # where a curious user might legitimately type "ignore case".
        weight = 1.0 if source == "retrieved_context" else 0.6
        return InjectionSignal(detected=bool(hits),
                               confidence=min(1.0, len(hits)*0.35*weight),
                               patterns=hits, source=source)
```


[illegible]

Section 3 — PII in Prompts, Logs, and Caches

#	Path PII Accumulates	Note
1	The user's question	"why was jane.doe@corp.com refunded?"
2	Retrieved sample rows	Day 19's get_sample_rows returns REAL DATA
3	Schema metadata	Column comments containing examples
4	Query results in context	Multi-turn carries rows forward
5	LOGS	The most-forgotten path, and the widest
6	CACHES	Day 24's semantic cache stores questions

Path 2 is the one that surprises people: get_sample_rows exists to help the model understand data formats, and it does that by putting real customer data into a prompt that may leave your perimeter.

```
class PIIHandler:
    def redact(self, text: str) -> tuple[str, dict]:
        """ONE-WAY redaction for logs and caches. Use when the value is
        never needed again."""

    def tokenize(self, text: str, trace_id: str) -> str:
        """
        REVERSIBLE tokenization for the PROMPT path.
        The model sees a stable token, generates SQL referencing that token,
        and the token is substituted back AT EXECUTION TIME – so the real
        value NEVER REACHES THE MODEL AT ALL.

        Input:          "orders for jane.doe@corp.com"
        Model sees:     "orders for <EMAIL_a3f9>"
        Model writes:   WHERE customers.email = '<EMAIL_a3f9>'
        Executor:       substitutes the real value from the vault
        """
        for f in sorted(self.detector.analyze(text), key=lambda x: -x.start):
            value = text[f.start:f.end]
            token = f"<{f.entity_type}_{hashlib.sha256(value.encode()).hexdigest()[:4]}>"
            self.vault.put(trace_id, token, value)    # stays SERVER-SIDE
            out = out[:f.start] + token + out[f.end:]
        return out

class SampleRowSanitizer:
    """
    Day 19's get_sample_rows is the HIGHEST-RISK tool in the toolkit,
    because its entire purpose is putting real rows into a prompt.

    THE FIX IS ARCHITECTURAL, NOT A FILTER:
    the model needs to know the FORMAT of the data, not its CONTENT.
    Synthesize a format-preserving example instead of returning real rows.
    """
    @staticmethod
    def _format_example(value) -> str:
        """Preserve the SHAPE, discard the CONTENT."""
        if value is None: return None
        s = str(value)
```

```
if "@" in s:         return "user@example.com"
if s.replace("-", "").isdigit() and len(s) >= 9:
    return "XXX-XX-" + s[-4:]           # last-4 pattern preserved
return "".join("9" if c.isdigit() else
               "x" if c.isalpha() else c for c in s)
```

■■■ LOG HYGIENE – the widest and most-forgotten exposure ■■■■■■■■■■■■■■■■■■■■

NEVER LOG (in ANY environment, including dev):

- Raw user questions containing PII -> log the REDACTED form
- Sample rows / query result sets -> log row COUNT and column NAMES
- Full prompt text with data inlined -> log the artifact VERSION (Day 23)
- Values in generated SQL -> log SQL with literals redacted

ALWAYS LOG:

- trace_id, tenant_id, user_id (pseudonymous ID, NOT an email)
- artifact_versions (Day 23 – reproducibility)
- guardrail decisions and reasons
- token counts and cost (Day 24 – attribution)
- SQL structure with literals replaced by placeholders

THE TEST THAT CATCHES MOST VIOLATIONS:

Run your PII detector across a day of PRODUCTION LOGS. Any hit is a finding. Teams that have never run this test invariably have hits – most commonly in ERROR PATHS, where an exception handler logs the full request context on failure. The happy path is usually clean because someone thought about it; the exception path never is.

■■■ CACHE HYGIENE (Day 24's semantic cache) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

A question containing PII becomes PII AT REST, in a system with a TTL rather than a retention policy.

RULES:

- Tokenize BEFORE embedding (the vault mapping is not cached)
- Cache the SQL SHAPE, never the result rows
- RIGHT-TO-ERASURE must be able to purge a user's cache entries -> requires a user_id index on the cache, which you must DESIGN IN UP FRONT because you cannot retrofit it onto an embedding store

Section 4 — Tenant Isolation and Data Residency

Day 20 mentioned tenant isolation. Here it is as a security property with adversarial testing, because "we pass tenant_id everywhere" is a claim, not evidence. A tenant-isolation failure is a reportable data breach, not a bug.

[illegible]

```
class ResidencyPolicy:
    tenant_id: str
    allowed_regions: set[str] # {"eu-west-1"}
    allow_managed_api: bool = False # does data leave your infra?
    allow_cross_border: bool = False
    retention_days: int = 30

class ResidencyEnforcer:
    def select_backend(self, policy, available) -> dict:
        eligible = [b for b in available
                     if b["region"] in policy.allowed_regions
                     and (policy.allow_managed_api or b["type"]=="self_hosted")]
        if not eligible:
            # FAIL CLOSED. NEVER silently fall back to a non-compliant
            # backend – that converts a CAPACITY problem into a
            # REGULATORY one.
            raise ComplianceError(
                f"No compliant backend for tenant {policy.tenant_id} "
                f"in {policy.allowed_regions}. Refusing to serve.")
        return min(eligible, key=lambda b: b["current_load"])

# NOTE FOR DAY 22's TOPOLOGY 4 (hybrid with fallback):
# The FALLBACK backend must be residency-checked too. A circuit breaker
# that fails over from an EU self-hosted pool to a US managed API is a
# CROSS-BORDER TRANSFER executed automatically at 3am by a component
# that has no idea it just did that.
# Wire ResidencyEnforcer INTO the BackendRouter, not alongside it.
```


Section 6 — Supply Chain and Model Provenance

The model itself is a dependency you did not compile, cannot read, and usually cannot fully verify. Treat it with the same seriousness as any other third-party binary in your build.

[illegible]

Section 7 — FAANG Security Practice

Google — Secure AI Framework (SAIF)

Google published SAIF as a conceptual framework for securing AI systems, organized around EXTENDING EXISTING SECURITY CONTROLS to AI rather than inventing a parallel discipline. Its most useful contribution for a practitioner is the emphasis on expanding DETECTION AND RESPONSE to cover AI-specific behavior, and on automating defenses to keep pace with new threats — which maps directly onto Section 2's argument that consequence-oriented guardrails outperform technique-specific injection detection. Google's framing that AI systems should INHERIT an organization's existing security controls rather than be exempted from them is the correct posture for the capstone: the database grant, the network egress policy, and the audit log are ORDINARY CONTROLS applied to an unusual workload.

Microsoft — Prompt Shields and the AI Red Team

Microsoft ships prompt-injection detection as a callable service in Azure AI Content Safety, with DISTINCT HANDLING for direct attacks on user input and indirect attacks embedded in documents — the same split as Section 1. Separately, Microsoft's AI Red Team has published on adversarially probing AI systems before deployment, noting that responsible-AI failures and security failures require different expertise and different test approaches. The structural lesson: injection detection is available as an off-the-shelf layer, but Microsoft's own documentation positions it as ONE DEFENSE AMONG SEVERAL rather than a solution — the same claim Section 2 makes about InjectionDetector.

Amazon — Bedrock Guardrails and IAM as the Real Boundary

Bedrock Guardrails provides content filtering, denied topics, and PII detection with redaction applied AROUND the model rather than INSIDE the prompt, and Bedrock integrates with IAM so that model invocation permissions, VPC endpoints, and KMS encryption are ORDINARY AWS ACCESS CONTROLS. This is the clearest commercial expression of Section 2's Layer 0 argument: the boundary that matters is the IDENTITY AND NETWORK layer, and the guardrail is defense in depth on top of it. For a Text-to-SQL system the analogous statement is that llm_query_role and row-level security ARE the security architecture; everything in the application is a supplement.

Meta — Llama Guard, Prompt Guard, and CyberSecEval

Meta's Purple Llama effort ships safety tooling as SEPARATE SMALL MODELS rather than as prompt instructions: Llama Guard classifies prompts and responses against a hazard taxonomy, Prompt Guard targets injection and jailbreak detection specifically, and CyberSecEval benchmarks security-relevant model behavior including susceptibility to prompt injection. The architectural pattern is the one Day 24 exploited for COST — a small dedicated model running alongside the large one — applied here to SECURITY. A 7B classifier screening inputs and outputs costs a fraction of the 70B generation call it protects, making the safety pass essentially free relative to the request.

Netflix — Least Privilege and Continuous Verification

Netflix's security engineering writing has long emphasized PAVED-ROAD tooling that makes the secure path the easy path, and CONTINUOUS AUTOMATED VERIFICATION of access rather than point-in-time review. Applied to LLM systems, continuous verification is what turns Section 4's IsolationTestSuite from a one-time penetration test into a CI GATE that runs on every change — which matters because tenant isolation regressions are introduced by ORDINARY REFACTORS far more often than by attackers. The paved-road idea shows up as SecureConnectionFactory: if the only convenient way to get a database connection is the one with the restricted role and tenant context set, developers do not have to REMEMBER to do it right.

Section 8 — Architect's Lens

THE CONTROL HIERARCHY — ranked by what survives a bad day

TIER 1 — SURVIVES ANY APPLICATION BUG

Database GRANTS (SELECT only, no auth tables)
 Row-level security policies
 statement_timeout at the engine
 Network egress restrictions on the inference host
 -> These hold even if EVERY LINE of your Python is wrong.

TIER 2 — SURVIVES MODEL MISBEHAVIOR

Day 22's guardrails (read-only, tenant scope, PII, cost, row cap)
 PII tokenization before the prompt
 Residency enforcement at routing time
 -> These hold even if the model is FULLY COMPROMISED by injection,
 because they inspect the OUTPUT rather than trusting the PROCESS.

TIER 3 — REDUCES ATTACK SUCCESS RATE

Trust boundary tags in the prompt
 Injection pattern detection
 Sample row sanitization
 -> Valuable, measurable, and NEVER SUFFICIENT ALONE.

TIER 4 — DETECTION AND FORENSICS

Audit log with hash chain | Security decision logging
 Isolation tests in CI
 -> Do not PREVENT anything. Determine WHETHER something happened,
 and whether it happened BEFORE.

THE FAILURE MODE TO AVOID: building TIER 3 FIRST because it is the part that feels like "AI security," and leaving TIER 1 as an open checklist item — which is exactly where Day 21's review left it.

WHY PROMPT INJECTION IS NOT "SOLVED" AND WON'T BE

SQL injection was solvable because SQL has a GRAMMAR permitting a hard separation of code from data — that is what a prepared statement IS. Natural language has no such grammar. There is no parameterized prompt, and there cannot be one while the interface is natural language.
 -> THEREFORE: ASSUME INJECTION SUCCEEDS. Design so that a fully compromised model still cannot do anything harmful. That assumption changes every design decision, and it is the correct one.

THE QUESTION THAT EXPOSES A WEAK DESIGN

"Assume the model is entirely under the attacker's control and will emit whatever SQL they choose. What is the worst outcome?"
 If the answer involves data loss, cross-tenant access, or credential exposure, your controls are in the WRONG TIER. The answer should be: "a blocked query and a logged security event."

COMPLIANCE OBLIGATIONS THAT SHAPE ARCHITECTURE, NOT JUST POLICY

Right to erasure -> cache/memory need a user_id index FROM DAY ONE;
 audit records must be PII-free by construction
 Data residency -> ROUTING DECISIONS, not a deployment note
 Purpose limitation -> per-feature cost/audit attribution (Day 24) is also
 the evidence for what data was used for what
 Explainability -> Day 18's citations + Day 23's version stamps are
 together what let you explain a historical answer
 Each is CHEAP on day one and EXTREMELY EXPENSIVE to retrofit.

WHAT TO DO FIRST, IN ORDER

1. Create `llm_query_role` with SELECT-only grants (hours, Tier 1)
2. Enable row-level security on tenant tables (hours, Tier 1)
3. Add `IsolationTestSuite` to CI (a day, Tier 4 -> gate)
4. Run your PII detector over production logs (an hour, finds real leaks)
5. Sanitize `get_sample_rows` (a day, Tier 2)
6. Everything else

Section 9 — Exercises and Summary

Exercise 1: Run the Indirect Injection Test

```
# In a NON-PRODUCTION database, set a column comment containing an
# instruction-shaped payload targeting a table outside the allow-list.
# Re-index (Day 16), then ask an ordinary question that retrieves it.
# Record: did the model comply? Did assert_tenant_scope BLOCK the result?
# The point is NOT whether the model resisted – it is whether the BLOCK
# HELD when the model did not. Tier 2 is what you are testing.
```

Exercise 2: Audit Your Logs for PII

```
# Run a PII detector across ONE DAY of your own application logs.
# Count hits by log statement. Expect the EXCEPTION HANDLERS to dominate:
# the happy path is usually clean because someone thought about it,
# and the error path is usually not because nobody did.
# Fix the top 3 offenders and re-run to confirm zero.
```

Exercise 3: Prove Tenant Isolation

```
# Implement all FIVE IsolationTestSuite tests against your system.
# Run them. If any PASSES on the first attempt without you having built
# the corresponding control, VERIFY THE TEST ACTUALLY EXERCISES THE PATH –
# a test that cannot fail is worse than no test.
# Then wire the suite as a REQUIRED CI CHECK (Day 23's gate pattern).
```

Exercise 4: Write the Worst-Case Statement

```
# In one paragraph: assume an attacker FULLY CONTROLS the model's output.
# Enumerate what they can and cannot cause, citing the SPECIFIC control
# that stops each item and WHICH TIER it is in.
# If any item is stopped ONLY by a Tier 3 control, that is your next piece
# of work. This paragraph is also the artifact a security reviewer will
# ask for, so writing it is not overhead.
```

Summary

Concept	Key Point
Three structural differences	No instruction/data boundary; writes+executes code; non-deterministic
Architectural consequence	Every real control sits OUTSIDE the model
Direct injection	User is attacker; risk overstated — they are already authenticated
Indirect injection	Data is attacker; persistent, targets other users; the serious one
The capstone's exposure	A column comment retrieved as schema context becomes trusted
InjectionDetector	Detection, NOT prevention; a signal for logging and rate-limiting
Context weighting	An imperative in retrieved data is far more suspicious
Layer 0 (strongest)	llm_query_role, SELECT-only grants, RLS, statement_timeout
Row-level security	The model cannot be tricked into access it cannot perform

SecureConnectionFactory	Tenant context set transaction-locally, never connection-level
Layer 1	Trust boundary tags — a mitigation, never the only control
_neutralize()	Prevents tag-boundary escape from retrieved content
Layer 2	Detect the CONSEQUENCE, not the technique — works on unknown attacks
Layer 3	Blast radius: no grant on auth tables, row caps, egress restrictions
Six PII paths	Question, sample rows, schema metadata, results, logs, caches
PIIHandler.tokenize	Model sees a token; the real value never reaches it
SampleRowSanitizer	The model needs FORMAT, not CONTENT — synthesize, do not filter
Log hygiene	Exception paths are where PII leaks; the happy path is usually clean
Cache hygiene	Right-to-erasure needs a user_id index designed in from day one
IsolationTestSuite	Five adversarial tests in CI — isolation breaks via refactors
Five leak points	Vector store, cache, memory, error messages, prompt cache
ResidencyEnforcer	Fail closed; wire INTO the BackendRouter, not alongside it
Fallback residency trap	A 3am circuit-breaker failover can be a cross-border transfer
AuditRecord	Who, what, how (artifact versions), decisions, outcome — PII-free
Hash chain	Turns "trust our database" into "verify the chain"
Erasure vs immutability	Contradictory unless audit records are redacted by construction
Two retention clocks	Audit long and PII-free; operational short
ModelSupplyChain	Checksum, serialization format, license, explicit approval
Serialization risk	Legacy pickle checkpoints execute code on load
Control hierarchy	Tier 1 survives app bugs; Tier 3 only reduces success rate
Assume injection succeeds	No parameterized prompt; design for a compromised model
The exposing question	"If the attacker controls the model, what is the worst outcome?"

NEXT: Day 26 — Multi-Modal Systems

Vision-language models, document understanding pipelines, image and PDF ingestion into RAG, and extending the capstone to answer questions over charts and scanned documents.